

Cartly

An Engineering Reconstruction of a Marketplace-Scale Shopping Experience

Amazon.com Clone — 24-Hour Software Engineering Challenge
Engineering Report

Author	Nitish R.G.
Date	3 October 2026
Repository	<code>github.com/NITISH-R-G/amazon-clone</code> (branch <code>main</code>)
Deployment	<code>amazon-clone-six-sigma-27.vercel.app</code> (Vercel, HTTPS)
Stack	Next.js 16, React 19, TypeScript (strict), PostgreSQL with Drizzle ORM, Stripe (test mode), Vitest, Playwright
Scope	A fictional marketplace: invented brands, sellers, products and reviews. No Amazon data or assets are used.

Reading guide. Throughout this report, claims are labelled by their evidence: **Implemented** (present in the source), **Tested** (covered by an automated test), **Not built** (considered or designed only) and **Not verified** (could not be confirmed from the repository).

Contents

Executive Summary	5
1 Problem Definition	7
2 Product Scope and Product Judgment	8
3 System Architecture	9
3.1 A modular monolith	9
3.2 Module dependencies	9
3.3 Boundaries that make it testable	10
3.4 Technology stack	11
3.5 Domain model	11
4 Catalogue, Variants and Marketplace	13
4.1 Catalogue and product model	13
4.2 The variant system	13
4.3 Marketplace offers and the buy box	14
5 Search, Discovery and Merchandising	15
5.1 Lexical retrieval	15
5.2 Structured filtering and facets	15
5.3 Hybrid retrieval: concept-based, not embedding-based	15
5.4 Homepage and merchandising	17
6 Pricing, Coupons and Delivery Promise	18
6.1 The pricing flow	18
6.2 Deals and coupons	18
6.3 The delivery promise	18
7 Inventory Consistency, Checkout and Payments	20
7.1 Why stock needs reservations	20
7.2 The reservation model	20
7.3 Three separate states	21
7.4 Two-phase checkout	22
7.5 Payment provider boundary	22
7.5.1 Stripe test mode	22
7.6 Cancellation and refunds	23
8 Recommendations, Sponsored Placements and Reviews	24
8.1 Recently viewed and personalisation	24
8.2 Deterministic recommendations	24
8.3 Sponsored placements	25
8.4 Reviews and trust	25

9 Customer Experience and Performance	26
9.1 Customer-facing decisions	26
9.2 Performance and scale	26
10 Testing, Security and Deployment	28
10.1 Testing and quality engineering	28
10.2 Security and trust boundaries	29
10.3 Deployment	29
11 Engineering Decisions, Limits and Future Architecture	31
11.1 Decisions and tradeoffs	31
11.2 What was deliberately not built	31
11.3 Limitations	32
11.4 Future architecture	32
12 Final Engineering Takeaways	34

List of Figures

3.1	Runtime view. Dashed boxes are external to the application.	9
3.2	Main module dependencies (arrow: “uses”). Not drawn, to keep it readable: checkout, discovery and reviews also read the catalogue, and the route layer uses auth for sessions.	10
3.3	Product, variant and offer. The variant row itself acts as the implicit first-party offer; seller offers are additional rows.	11
3.4	Simplified entity relationships. Solid arrows: references. Dashed: reservation belongs to an order by identifier.	12
4.1	From product page to cart line. The cart stores the variant and the offer, not a product.	13
5.1	Hybrid retrieval. The lexical path always runs first and is authoritative.	16
6.1	Order of computation (<code>checkout/internal/quote.ts</code>). Deals are already in the offer price as a list-price reduction.	18
6.2	Inputs to the delivery promise.	18
7.1	Two customers, one unit. The database lock serialises the reservations; only A can proceed to payment.	21
7.2	The three state machines (scaled to fit). The order also carries a separate refund status (pending, refunded, failed). Shipped and delivered are derived from time, not stored.	22
10.1	Path from development to production.	29

List of Tables

1	The project at a glance (all figures from the repository).	5
1.1	What a customer sees versus what must be true behind it.	7
2.1	Scope decisions: what was built, why, and what was deliberately deferred. . . .	8
3.1	Technologies actually used.	11
8.1	Recommendation signals (<code>discovery/score.ts</code>).	24
10.1	Test layers and results at the time of writing.	28
11.1	Principal decisions (numbers refer to the repository’s decision log).	31

Executive Summary

The challenge. Rebuild the Amazon.com shopping experience in roughly 24 hours (the repository history spans about 22 hours, from the first commit on 2 October to the last on 3 October 2026), and be judged on speed, product judgment and UX quality.

The objective we set. Do not produce a visual clone. The visible interface of a marketplace is the surface of a stateful system: a catalogue with typed attributes, variants that carry their own price and stock, competing seller offers, reservations that stop two customers buying the same last unit, a payment lifecycle that is separate from the order lifecycle, and ranking rules that decide what each customer sees. We built a small version of that system and made every customer-visible number (price, discount, delivery date, rating) a server-side computation.

What exists. A Next.js application organised as a modular monolith (nine modules with enforced boundaries) on PostgreSQL, with a 2,400-product synthetic catalogue, multi-dimension variants, 608 additional seller offers on top of an implicit first-party offer, a deterministic buy-box rule, PostgreSQL full-text and trigram search with structured facets, a concept-based hybrid retrieval layer, a two-phase checkout with stock reservations, Stripe test-mode integration behind a provider interface (PaymentIntent, Payment Element, signed webhook), server-side coupons, a deterministic delivery promise, recently-viewed and deterministic recommendations, labelled sponsored placements, and verified reviews.

What is verified. 131 unit and integration tests across 26 files pass; type checking and linting are clean; Playwright journeys cover desktop and a phone viewport, including a two-browser race for the last unit of stock. The application is deployed over HTTPS on Vercel, and the final commits are recorded as successful production deployments.

What is not verified. A completed card payment against the live Stripe API was not performed from this repository's evidence; Stripe behaviour is covered by unit tests with real signature verification, and a key-gated end-to-end test exists but was not run. The database host in production is configured through an environment variable and is not independently confirmed here.

Table 1: The project at a glance (all figures from the repository).

Products	2,400	Database tables	21
Product types	43	SQL migrations	20
Fictional brands	104	Documented decisions	32
Variants	8,860	Unit/integration tests	131 (26 files)
Seller offers	608	E2E scenarios	12 × 2 viewports
Application modules	9	Source under <code>src/</code>	≈13,900 lines

Main decisions. PostgreSQL for everything, including search (no dedicated search engine); deterministic ranking everywhere (no machine learning); a payment-provider interface with a demo and a Stripe implementation; reservation-based inventory; and a hard rule that client-supplied prices, discounts and “payment succeeded” signals are never trusted.

Main constraints. Twenty-four hours; no access to real Amazon data; no paid external ser-

vices beyond Stripe's free test mode. These drove the choices in Chapters 2 and 11.

1 Problem Definition

A basic storefront shows products, takes an order and sends an email. A marketplace like Amazon.com is a different kind of system, because almost every thing a customer sees is the output of a separate decision that must remain consistent with the others.

Table 1.1: What a customer sees versus what must be true behind it.

On the page	Behind it
A product with options	A <i>product</i> (the idea), <i>variants</i> (the exact purchasable configurations with their own SKU, price and stock) and <i>offers</i> (who sells that variant, at what price, from where).
“In stock”	A quantity that other customers are changing at the same moment, and that must not be sold twice.
A price	A seller’s price, a list price, a deal, a coupon, shipping rules and tax, all computed in one agreed order.
Search results	Text matching, typo tolerance, structured filters whose options depend on the kind of product, and a ranking.
“Arrives Tue, Oct 6”	A promise derived from fulfilment, handling time, cut-off and destination.
“Place order”	A payment lifecycle (pending, failed, succeeded, refunded) that is not the same thing as the order lifecycle (awaiting payment, placed, shipped, delivered, cancelled).
Stars and reviews	Aggregates that must match the review records, and a “verified purchase” flag that must mean something.

The central engineering thesis of this project follows from the table: *the visible shopping interface is only the surface of a much larger stateful marketplace system*. A visual copy of the pages would show none of the hard parts. We therefore chose to reproduce the hard parts at small scale and to be explicit about where our rules are our own rather than Amazon’s.

2 Product Scope and Product Judgment

With 24 hours, the question was never “what can we build” but “what must be true for a reviewer to believe this is a marketplace, and what can we leave out without losing that belief”. Table 2.1 records the answer. The ordering of work followed one rule from the project’s instructions: protect the purchase path (browse, variant, cart, checkout, order) first, and cut decoration before correctness.

Table 2.1: Scope decisions: what was built, why, and what was deliberately deferred.

Area	Implemented	Why it mattered	Deliberately deferred
Catalogue	2,400 synthetic products, 43 types with typed attributes, 104 invented brands	Scale and heterogeneity make search, facets and pagination real	Real product data and imagery (none was licensed)
Variants	Up to three dimensions, SKU per variant, price/stock/picture per variant	Variant identity is the core of “a product page”	Bundles, subscriptions
Sellers and buy box	608 seller offers, implicit first-party offer, deterministic selection rule	Shows that one product can have several sellers	Seller dashboards, seller ratings, onboarding
Search	PostgreSQL full-text and trigram, facets, typo tolerance, concept-based hybrid layer	Discovery is how customers find anything	Dedicated search engine, learned ranking
Inventory	Time-limited reservations with row locking and a concurrency test	Overselling is the classic marketplace failure	Warehouses, multi-location stock
Checkout and payments	Two-phase checkout, Stripe test mode behind a provider interface, idempotent webhooks, refunds	Trust: money must follow verified facts	Saved cards, live payments, fraud checks
Pricing	Deals, server-side coupons, shipping and tax rules	Totals must be explainable and tamper-proof	A general promotion engine
Delivery	Deterministic delivery windows on product, cart, checkout and order	“When will it arrive” drives purchase decisions	Carrier integration, tracking
Discovery	Recently viewed, deterministic recommendations, personalised home rails, sponsored placements	Makes the home page react to the customer	Machine-learned recommendation, ad auction
Reviews	Reviews with a verified-purchase flag derived from delivered orders	Trust signal with real semantics	Votes, images, moderation
Accounts	Register, sign in, guest cart merge, order history; guest checkout stays open	Accounts should add history, not gate purchase	OAuth, saved addresses, returns

Deferred items are not failures; each was weighed against its cost in the time box and against whether its absence would change what a reviewer could verify. The recorded reasoning is in the project’s decision log (D1–D32).

3 System Architecture

3.1 A modular monolith

The application is one deployable Next.js program organised into modules. Each module exposes a small public interface (`index.ts`) and keeps its implementation in an `internal/` directory; a lint rule forbids importing another module's internals. This gives most of the design discipline of services without the operational cost, which suited a one-person, one-day build.

Implemented

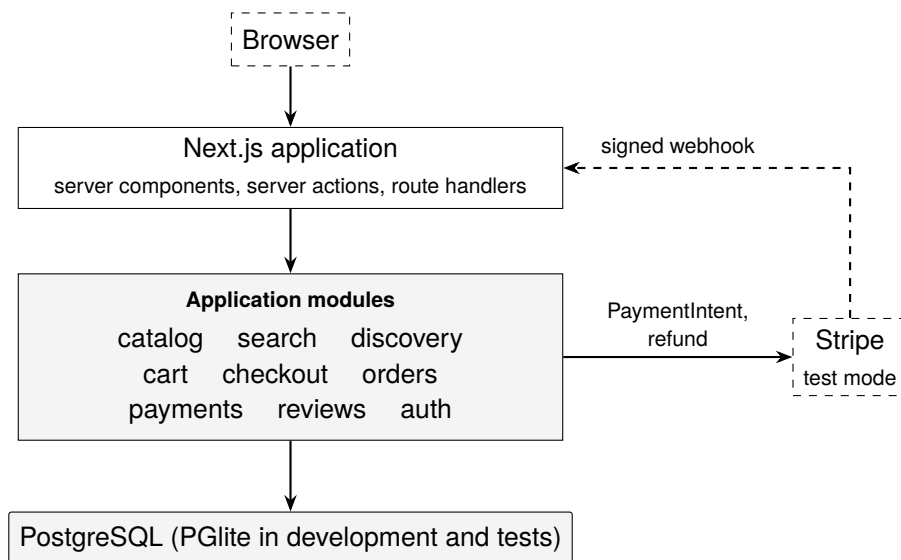


Figure 3.1: Runtime view. Dashed boxes are external to the application.

3.2 Module dependencies

Figure 3.2 shows who may call whom. Checkout is the only orchestrator: it asks the cart for lines, the catalogue to reserve stock, payments to talk to the provider and orders to persist the result. The composition root (`src/server/app.ts`) is the only file that wires modules together.

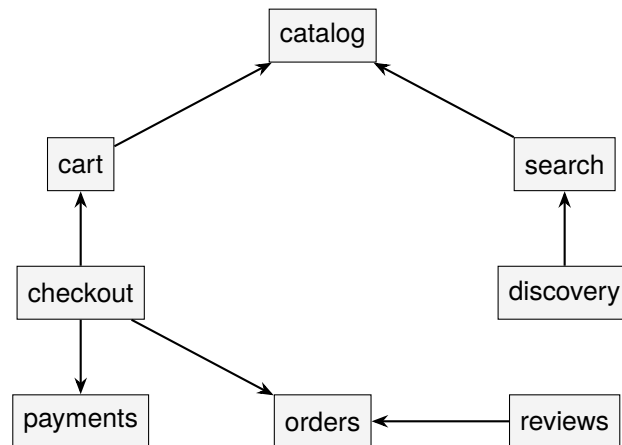


Figure 3.2: Main module dependencies (arrow: “uses”). Not drawn, to keep it readable: checkout, discovery and reviews also read the catalogue, and the route layer uses auth for sessions.

3.3 Boundaries that make it testable

Only three things are replaced by test doubles: the *clock*, the *identifier generator* and the *payment provider*. The database is always real: tests run a genuine PostgreSQL engine in-process (PGlite), with the same migrations as production. This is deliberate. The interesting failures in this system (a lock that does not serialise, a constraint that does not hold) disappear if the database is mocked. **Implemented Tested**

3.4 Technology stack

Table 3.1: Technologies actually used.

Technology	Role	Reason
Next.js 16 (App Router)	Routes, server rendering, server actions, route handlers	One deployable unit; server components keep pricing and ranking on the server
React 19, TypeScript (strict)	UI and types across the whole codebase	Types carry the domain model (money is integer cents)
Tailwind CSS 4, shadcn/ui, Radix	Components and styling	Accessible primitives; restrained monochrome design
PostgreSQL	System of record <i>and</i> search engine	Transactions, row locks, full-text, trigram; one database to operate
Drizzle ORM	Schema and typed queries, SQL migrations	Schema in TypeScript; plain SQL where needed
PGLite	PostgreSQL in-process for development and tests	Fast, real engine; same SQL as production
pg	Production driver	Pooled connections to managed PostgreSQL
Stripe (SDK, Payment Element)	Test-mode payments	Real PaymentIntent lifecycle and signed webhooks
Zod	Input validation at boundaries	Shared parse-then-trust pattern
Vitest, Playwright	Unit/integration and end-to-end tests	Real database in tests; desktop and phone viewports
Vercel	Hosting over HTTPS	Git-connected deployment; migrations run at build time
pnpm, ESLint	Tooling	Reproducible installs; module-boundary lint rule

3.5 Domain model

The most important idea in the model is the three-level distinction between product, variant and offer (Figure 3.3). A *product* is what a customer thinks of as “the laptop”. A *variant* is one exact configuration of it with its own SKU. An *offer* is a particular seller’s way of selling that exact variant. As an illustration (not a claim about a seeded item): the product “Laptop X” has the variant “16 GB / 512 GB / Silver”, and “Seller A” may offer that variant at a given price and shipping cost while another seller offers the same variant for less but ships later.

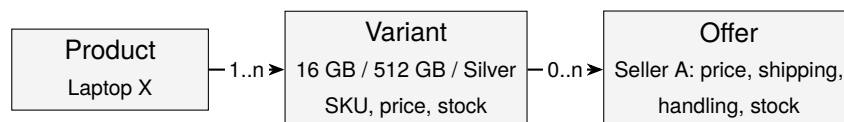


Figure 3.3: Product, variant and offer. The variant row itself acts as the implicit first-party offer; seller offers are additional rows.

Figure 3.4 shows the main tables (21 in total; satellite tables such as sessions and campaigns omitted for readability).

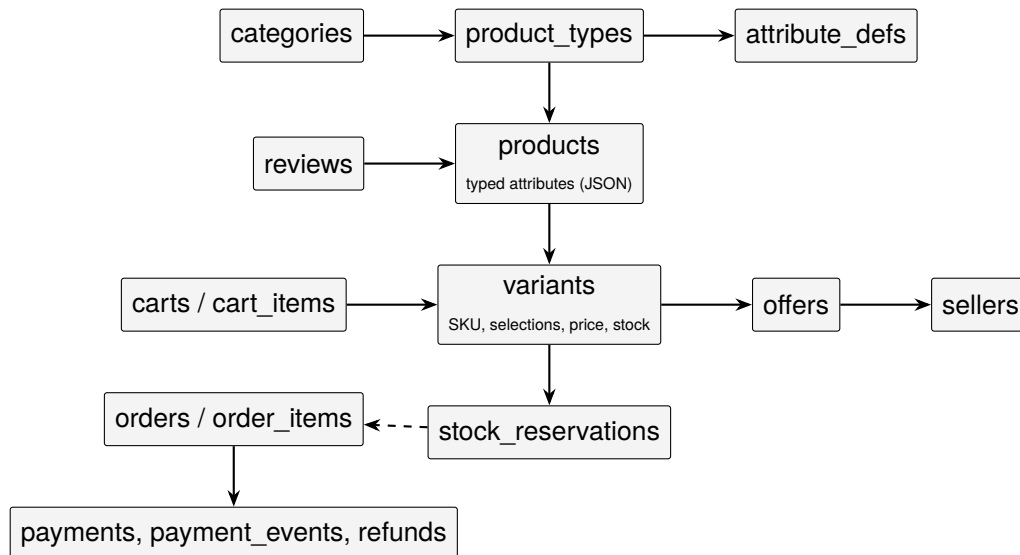


Figure 3.4: Simplified entity relationships. Solid arrows: references. Dashed: reservation belongs to an order by identifier.

Other tables: `promotions` (coupons), `product_views`, `sponsored_campaigns`, `users` and `sessions`. Money is stored as integer cents throughout, and order lines are purchase-time snapshots (title, price, SKU, seller) so that later catalogue changes never rewrite history.

4 Catalogue, Variants and Marketplace

4.1 Catalogue and product model

The seeded catalogue has 2,400 products: 30 hand-written (they carry the featured placements) and 2,370 generated deterministically from a seeded pseudo-random generator, so every environment gets the same data. They belong to 43 product types across nine departments, and carry 104 invented brand names. All pictures are flat illustrations generated by a script in the repository. **Implemented**

Products are not uniform records. Each *product type* owns a set of *attribute definitions*, each marked as a *variation* attribute (what variants differ by: colour, storage, RAM) or a *spec* attribute (a technical detail: refresh rate, processor), and optionally as a filter facet. A smartphone therefore has storage and RAM; a running shoe has size, colour, cushioning and terrain. A product stores its spec values as typed JSON keyed by attribute; the type supplies the vocabulary and labels. This lets the product page render a technical table and the search page offer the right filters without special-casing categories. **Implemented**

4.2 The variant system

A variant is an exact purchasable configuration. It has a SKU, its *selections* (for example colour, storage and RAM), a price, a list price, a stock count and, when colour differs, its own pictures. Of the 8,860 seeded variants, many products have three variation dimensions. Only valid combinations exist: a 1 TB option is not offered with 8 GB RAM if the catalogue does not sell it. **Implemented**

On the product page the customer picks values dimension by dimension. A pure function (`catalog/variants.ts`, importable from client components) resolves the selection to a variant; when the exact combination does not exist, it moves to the nearest valid variant instead of showing a dead end. Price, stock, SKU and picture follow the resolved variant. **Tested** (variant resolution is unit-tested.)

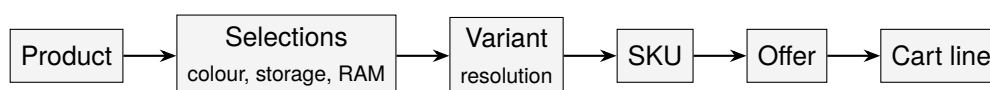


Figure 4.1: From product page to cart line. The cart stores the variant and the offer, not a product.

Why identity must survive. The cart line stores the variant identifier and the offer identifier; checkout re-reads both; the order snapshots SKU, option label and seller. Two configurations of the same product are therefore two cart lines, and a line is distinguished from another by variant *and* offer. The customer can see which configuration they are buying at every step, and the order still describes what was bought if the catalogue later changes. **Implemented Tested**

4.3 Marketplace offers and the buy box

Every variant has an implicit *first-party* offer (the variant's own price and stock, sold and fulfilled by "Cartly"). The `offers` table adds 608 seller offers, each with its own price, shipping charge, handling time, fulfilment mode (Cartly-fulfilled or seller-fulfilled) and stock. A cart line records which offer it came from, so the same variant from two sellers is two lines, stock is taken from the chosen offer, and a seller line carries its own shipping. **Implemented Tested**

The *buy box* is the offer pre-selected on the page. Our rule is our own and is not Amazon's proprietary algorithm:

1. Consider only offers with stock; if none, show the first-party offer as unavailable.
2. Compute each offer's *landed price*: price plus shipping. Offers within 2% of the lowest landed price are contenders.
3. Among contenders prefer Cartly-fulfilled over seller-fulfilled, then the first-party offer, then shorter handling time, then lower landed price, then offer identifier.

The final tie-break on identifier makes the outcome independent of the order in which offers happen to be read. The product page shows the selected seller, fulfilment, price, availability and delivery window, and lists the others under "Other sellers" with the same facts so that choosing one is an informed choice rather than noise. The seller's handling time also pushes the delivery window back (Chapter 6). **Tested** (representative tests: tie-breaks, 2% tolerance, order independence.)

Listing 4.1: The contender rule, simplified from `catalog/offers.ts`.

```
const lowest = Math.min(...buyable.map(landed));
const contenders = buyable.filter((o) => landed(o) <= lowest * 1.02);
// rank: Cartly-fulfilled, first-party, handling time, landed price, offer id
```

5 Search, Discovery and Merchandising

5.1 Lexical retrieval

Search is a function of the catalogue module (`catalog.findProducts`) running against PostgreSQL. Each product row carries a generated full-text vector indexed with GIN; title and brand also have trigram indexes. A query goes through three progressively looser readings and stops at the first that finds anything:

1. every word must be the start of a word in the product (or its category name);
2. the same, but a word may also be a close misspelling of a title or brand word (trigram word similarity above a threshold of 0.5, set per transaction so it never leaks across pooled connections);
3. only half of the words (rounded up) must match, and the page then says so explicitly.

Very common words are ignored so that nonsense such as “this is not a product” does not match everything, and a query made only of symbols returns nothing. Suggestions in the search box use the same machinery. **Implemented Tested**

5.2 Structured filtering and facets

Filters available on the results page are department, product type, brand, price range, minimum rating, in-stock only, on-sale, and the *attribute facets of the chosen product type* (for example processor or refresh rate once “Televisions” is chosen). Facet counts ignore their own filter, so a customer can see what choosing another value would yield. Sorting (relevance, price, rating, newest) and bounded pagination are server-side; all state lives in the URL so results can be linked, reloaded and navigated with the back button. No query loads the whole catalogue into memory. **Implemented Tested**

5.3 Hybrid retrieval: concept-based, not embedding-based

Accuracy note

The semantic layer is *concept-based*. It does not use embeddings, a vector database, a neural model or a language model. It is a curated lexicon that maps what a query means to the product types that satisfy it.

What it does. The lexicon contains 18 concepts. Each has trigger words and an ordered list of product types, for example the intent “keeping drinks hot or cold” (triggers such as *hot*, *insulated*, *commute*) maps to insulated water bottles, travel tumblers and gooseneck kettles. `interpretQuery` scores concepts by how many triggers match and returns up to four product types, deterministically. This lets the query “*something to keep my coffee hot on my commute*”, which shares no product words with an insulated bottle, return insulated bottles and travel tumblers. **Implemented Tested**

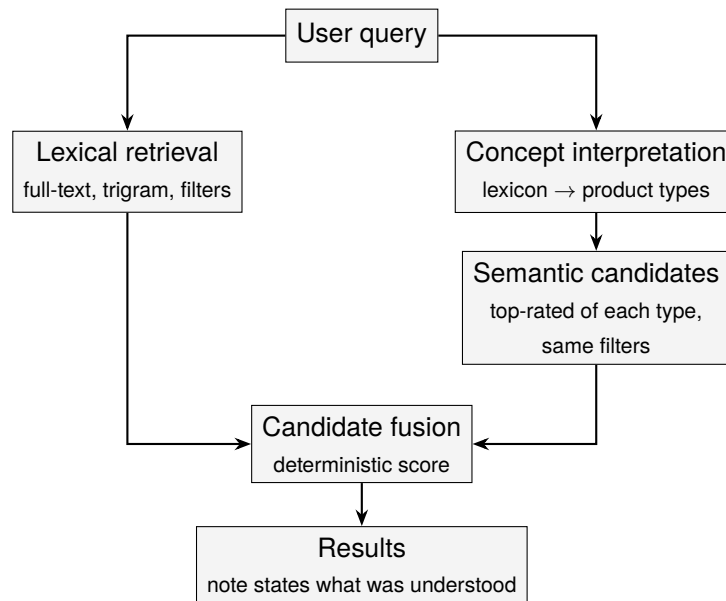


Figure 5.1: Hybrid retrieval. The lexical path always runs first and is authoritative.

When it runs. The concept layer is consulted only on the first page, with relevance sorting, when no product type has been chosen, and when the words alone found fewer results than a page or only a partial match. Exact queries (“studio headphones”) are unchanged. The semantic candidates are fetched with the *same* filters as the lexical ones, so price caps, brand and stock filters are respected.

Fusion. Each candidate list is already in rank order. The score is

$$\text{score} = w_{\text{lex}} \frac{1}{1 + r_{\text{lex}}} + w_{\text{sem}} \frac{1}{1 + r_{\text{sem}}} + w_{\text{biz}} \frac{\text{rating}}{5},$$

where a product absent from a list contributes zero for that term, and ties are broken by product identifier. When the lexical match was complete the weights are (0.6, 0.3, 0.1); when it was partial or empty, meaning is trusted more: (0.25, 0.65, 0.1). **Tested** (tests assert exact scores and orderings.)

Fallback. The whole semantic stage is wrapped so that any failure returns the plain lexical result. It depends on no external service.

Honest boundaries. The layer understands only what the lexicon covers. During customer-perspective testing, “*cheap thing for college*” returned laptops (the word “cheap” is not understood) and “*gift for my dad*” returned home-decor items. These are weak results of exactly the kind a learned model might improve, and they are recorded here rather than hidden. The validated demonstration query is the coffee-and-commute example above. Section 11.4 describes how this could evolve.

5.4 Homepage and merchandising

The home page is generated from data, not hand-placed. Each rail has a named source and ranking: “today’s deals” (products with a list price above the price, best rated), “top rated” (in stock, by rating), “featured” (curated rank), “new arrivals” (by creation date), “sponsored” (active campaigns, Chapter 8), and, once the visitor has browsing history, “pick up where you left off” (recently viewed) and “recommended for you” (Chapter 8). A rail with no products renders nothing. Category tiles come from the real department facet counts. **Implemented**

6 Pricing, Coupons and Delivery Promise

6.1 The pricing flow

Every amount the customer pays is computed on the server from the server-side cart. The browser never supplies a price, a discount or a total.

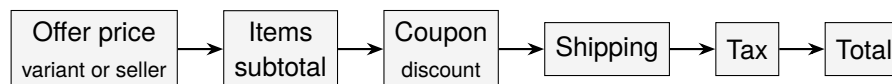


Figure 6.1: Order of computation (`checkout/internal/quote.ts`). Deals are already in the offer price as a list-price reduction.

The rules are provisional demo rules, isolated in one file: first-party items ship free from a \$35.00 subtotal, otherwise \$4.99; each seller line adds the shipping that seller charges; tax is 8% of the items *after* any coupon. All arithmetic is in integer cents, with tax rounded half up.

Implemented Tested

6.2 Deals and coupons

A *deal* is simply an offer whose list price exceeds its price; the interface shows the original price, the price and the percentage saved. A *coupon* is a row in the `promotions` table: a code, a kind (percent or fixed amount), a value, a minimum items subtotal and a validity window. Three demonstration codes are seeded: `SAVE10` (10% off, minimum \$50.00), `WELCOME5` (\$5 off, minimum \$25.00) and `SPRING20`, which is expired on purpose so the failure path is visible.

Validation is a pure function: the code is matched case-insensitively; it must have started and not ended; the items subtotal must reach the minimum; a percentage rounds down and a discount can never exceed the subtotal. The cart stores only the *code* (in a cookie); the discount is recomputed on every quote and again when checkout starts, and the order stores the code and the discount amount. A client can therefore not invent or alter a discount. The messages distinguish an unknown code, a code not yet active, an expired code and a minimum not reached, and the latter shows that code's own minimum. **Implemented Tested**

One limitation found during customer-perspective testing: the expired-code message does not state the expiry date.

6.3 The delivery promise

“When will it arrive” is answered by a deterministic function (`catalog/delivery.ts`) that takes the current time, the fulfilment mode, the seller's handling time and, when known, the destination ZIP code.

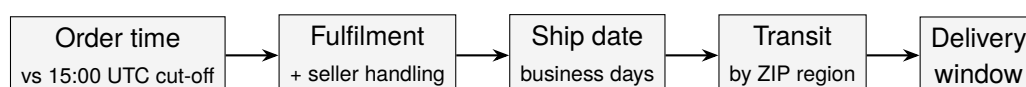


Figure 6.2: Inputs to the delivery promise.

The rule: orders placed before the cut-off on a weekday start processing that day, others on the next business day. Cartly-fulfilled offers ship on the processing day; seller-fulfilled offers add one business day plus one more for every 12 hours of the seller's stated handling time. Transit takes two, three or four business days depending on the first digit of the ZIP code (unknown: three), and the result is shown as a one-day window, for example "Wed, Oct 7 – Thu, Oct 8". Weekends are never delivery days. For a whole cart the slowest line decides. The same function produces the text on the product page, the other-sellers list, the cart, the checkout summary and the order. **Implemented Tested** (four tests cover the cut-off, weekends, seller handling and destination.)

This is a simulation of a promise, not a logistics system: there are no carriers, warehouses or tracking. The separate order status timeline (placed, shipped, out for delivery, delivered) runs on a compressed demo clock, and the report does not claim the two represent the same real-world process.

7 Inventory Consistency, Checkout and Payments

7.1 Why stock needs reservations

A naive checkout reads the stock, charges the card and decrements the stock at the end. Between the read and the write another customer can do the same, and the last unit is sold twice. Charging first and decrementing later is worse, because the failure appears after the money moved. This project instead *reserves* stock when checkout starts, converts the reservation into a sale only when payment is confirmed, and releases it on failure, expiry or cancellation.

7.2 The reservation model

Table `stock_reservations` holds one row per order, variant and offer with a quantity, an expiry time and a status: *held*, *committed* or *released*. A unique constraint on (order, variant, offer) makes reserving idempotent. A hold lasts 15 minutes. Expiry is evaluated lazily: a hold whose time has passed is simply ignored when availability is computed, so no background job is required. **Implemented**

To reserve, the module locks the stock row for the variant (or the offer, for seller lines), counts the unexpired holds of *other* orders, and refuses if the remaining stock is smaller than the request. Lines are processed in a fixed order to avoid deadlocks, and all lines succeed or none do. Because the stock row is locked first, two customers racing for the final unit are serialised by the database: the second one sees the first one's hold and fails. Commit re-checks the same condition, so a hold that expired and was taken by someone else cannot be committed. **Tested**

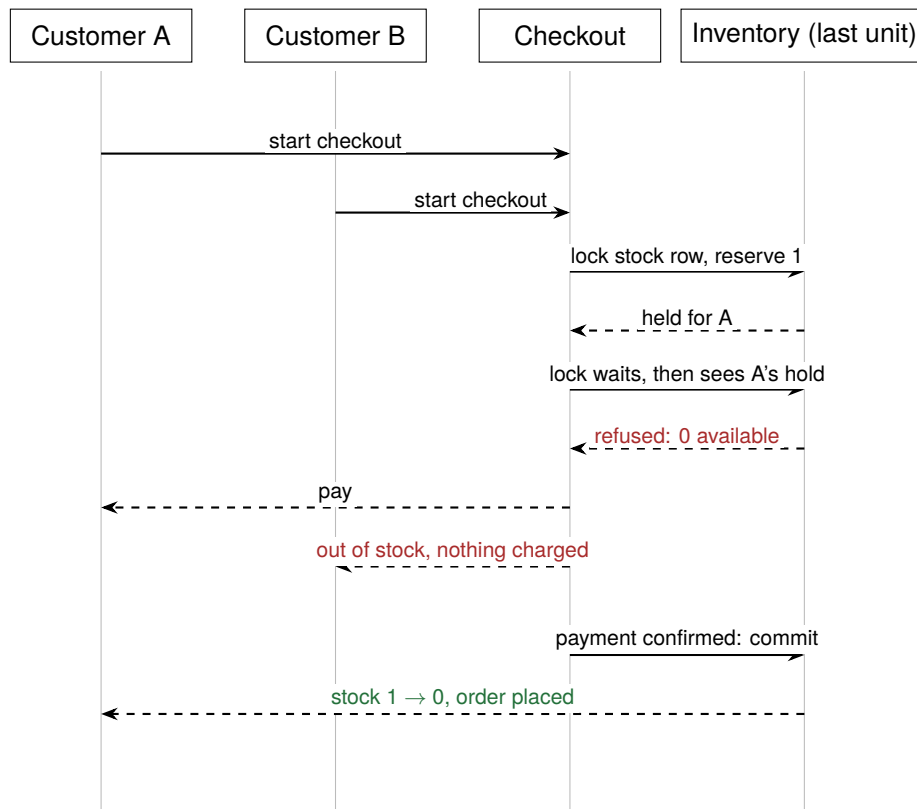


Figure 7.1: Two customers, one unit. The database lock serialises the reservations; only A can proceed to payment.

This is a single-database transactional design. It is correct for one PostgreSQL primary and would need a different mechanism (for example a dedicated inventory service) at multi-region scale; Chapter 11 says so.

Evidence. The module-level test starts both checkouts concurrently against the real database and asserts that exactly one succeeds; an end-to-end test drives two separate browser sessions through the checkout form at the same moment and asserts that exactly one sees “Order placed” and the other a sold-out message with nothing charged. Further tests cover expiry (a freed unit becomes buyable by the other customer), release on cancellation, and a late payment after expiry (the order is honoured if the unit is still free, otherwise cancelled and refunded).

Tested

7.3 Three separate states

The central design rule of the payment system is that *inventory, payment and order are three state machines that never share a field*. The order is not “paid” because a button was clicked; it is placed because the payment record says succeeded, and the stock is sold at that same moment.

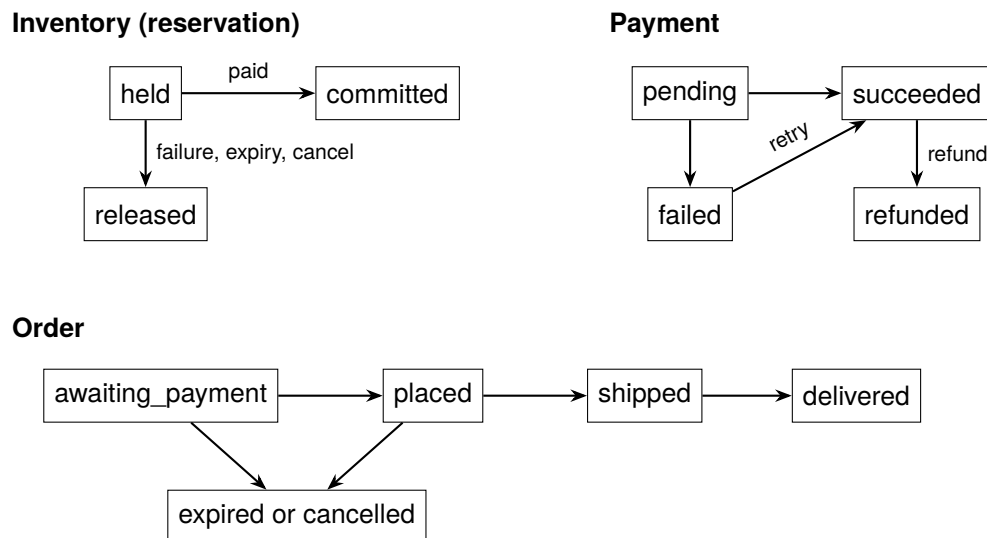


Figure 7.2: The three state machines (scaled to fit). The order also carries a separate refund status (pending, refunded, failed). Shipped and delivered are derived from time, not stored.

7.4 Two-phase checkout

Phase 1: start checkout (one database transaction). Validate the address; take an advisory lock on the customer and idempotency key; re-read the server cart and price it, including any coupon; abandon the customer's other unpaid orders; create the order in the *awaiting payment* state with purchase-time snapshots; reserve stock. Then, outside the transaction, ask the payment provider to create the payment. Starting again with the same key returns the same order; if the cart has changed meanwhile it is refused with a clear message rather than silently repriced. The cart is *not* cleared yet. **Implemented Tested**

Phase 2: payment event (one transaction). A provider event is recorded and de-duplicated, the pure state function updates the payment, and only if the payment is now *succeeded* does checkout commit the reservations, mark the order paid (this starts the fulfilment clock), and clear the cart. If the order was cancelled meanwhile, or the held units can no longer be committed, the order is cancelled and the money is refunded instead of being double-sold.

7.5 Payment provider boundary

Checkout depends on a `PaymentProvider` interface: create a payment, retrieve it, cancel it, refund it, verify a webhook, and (demonstration provider only) accept card details. Nothing in the interface returns “the customer paid”; success can arrive only as a verified event or a server-side retrieval. Two implementations exist: a demonstration provider that simulates a bank (used locally and in tests) and a Stripe implementation in test mode. The application selects Stripe when a Stripe secret key is configured. **Implemented**

7.5.1 Stripe test mode

On the server, the order total is used to create a `PaymentIntent` with the order identifier in its metadata and an idempotency key. The browser receives only the client secret and the publishable key and renders Stripe's Payment Element; card data never touches the application

server. After confirmation Stripe redirects the customer to the order page, but that redirect is not trusted. The order is placed by one of two server-side paths:

1. **Signed webhook.** A route handler reads the *raw* request body, verifies the signature with the webhook secret, and maps recognised events (payment succeeded, failed, processing, requires action, canceled, charge refunded) into the application's own event type. Unrecognised events and events for unknown payments are acknowledged without effect; a bad signature is rejected; an internal error returns a failure so Stripe retries.
2. **Server-side retrieval.** The payment and confirmation pages ask Stripe for the `PaymentIntent` and handle the answer exactly like an event, so a delayed or missing webhook cannot strand a paid order.

Idempotency and ordering. Events are recorded by provider event identifier, so a replay changes nothing. A pure state function accepts success from any open state, ignores events older than the last applied one, and allows refunds only after success; therefore a late “failed” event cannot undo a success. A refund row is written as pending *before* the provider is called, its identifier is the provider's idempotency key, and the payment counts as refunded only when the provider says the refund succeeded. A failed refund is shown as failed and can be retried; the interface never claims a refund that has not happened.

Listing 7.1: Rule from the payment state function, simplified.

```
// success wins from any non-final state; other changes only while open and not older
if (event.type === "payment.succeeded" && !isFinal(state)) return succeeded;
if (isOlder(event, state.lastEventAt)) return unchanged;
```

Evidence and its limits. The webhook path is covered by five tests that use the real Stripe signature verification with generated signatures (replay, tampered body, missing signature, out-of-order events, unknown payment, refund after cancellation); the state machine and checkout flow are covered by the payment-flow tests. **Tested** A completed card payment against the live Stripe API from this repository was *not* performed or recorded; the key-gated browser test for it exists but was not run. **Not verified** In production the checkout button reads “Continue to payment”, which indicates a Stripe key is configured, but successful settlement is unconfirmed.

7.6 Cancellation and refunds

Cancellation is allowed while an order is awaiting payment (the hold is released and the payment cancelled) and, for a paid order, only during the first minutes while it is still “placed”. A paid cancellation restocks the units in the same transaction, then requests a refund from the provider and records the outcome separately from the order status.

8 Recommendations, Sponsored Placements and Reviews

8.1 Recently viewed and personalisation

Every product page view is recorded against the visitor (signed-in user, or an anonymous browser identifier) in `product_views`, one row per visitor and product, newest first. The anonymous identifier lives in its own cookie, separate from the cart's guest token: an early version recorded views with the cart token and could race with cart creation, so the two were separated. On sign-in the anonymous history is merged into the account. The home page uses it to show "Pick up where you left off" and "Recommended for you"; with no history, those rails are simply absent and the generic rails remain. **Implemented Tested**

8.2 Deterministic recommendations

Recommendations are a transparent weighted sum computed from a bounded candidate pool (the best-rated products of each seed's type and category). For a seed product and a candidate:

Table 8.1: Recommendation signals (`discovery/score.ts`).

Signal	Contribution
Same product type	+4
Same category (different type)	+2
Same brand	+2
Shared attribute values	+0.5 each, at most 3
Price proximity (log scale)	0 to +2
Rating	0 to +1.5
Popularity (rating count, capped at 1,000)	0 to +1
On sale	+0.75

With several seeds (the last three viewed products) each seed's similarity is divided by its recency rank, so the most recent view counts most. Seeds and out-of-stock products are never recommended, and ties break on product identifier. The same function powers "Related products" on a product page (one seed) and "Recommended for you" (up to three seeds). **Implemented Tested**

We chose a transparent rule over a trained model because the time box allowed no data to train on, because a rule can be explained to a customer and a reviewer, because it is reproducible (the same inputs always give the same page, which makes it testable), and because it needs no infrastructure. It is not machine learning and the report does not call it that.

8.3 Sponsored placements

A sponsored campaign is a row in `sponsored_campaigns`: a product, a placement (search or home), keywords, a bid, a budget and a date window. The service returns active campaigns with budget remaining, ordered by bid (ties by identifier). For search placements the campaign's keywords must match the query; products already in the organic results are excluded so a product never appears twice.

The crucial property is separation: sponsored products are returned by a separate call and rendered in their own block labelled "Sponsored"; they never enter, reorder or alter the organic ranking, which a test asserts by comparing organic results before and after. A sponsored product that is out of stock is not shown. This is an observable simulation of marketplace advertising, not an ad auction: there is no pricing mechanism, spend accounting or relevance model. **Implemented** **Tested**

8.4 Reviews and trust

A review stores a product, an optional variant, an author, a rating from one to five, a title, a body, a "verified" flag and a timestamp. Three properties matter.

- **Verified means something.** The flag is set only by the server, and only when the signed-in user has a *delivered* order containing a variant of that product; a client cannot claim it. A user may review a product once, and an order that is merely placed does not qualify. **Tested**
- **Aggregates are real.** The displayed rating and count of a product are recomputed from its review rows (at seeding, and after each new review), and the page shows the average, the count, a five-bar distribution and the review text with a verified badge.
- **Seeded data is labelled honestly.** The catalogue is seeded with two to eight invented reviews per product written from templates and a seeded random generator, with some flagged verified. These flags are synthetic data, not outcomes of the eligibility rule. The demonstration account (credentials documented in the repository for demo use) has three delivered orders so that the real rule can be shown live.

Verified reviews are more meaningful than arbitrary star averages because they connect an opinion to a completed transaction. In this project that connection is real for new reviews and simulated for seeded ones, and the report keeps that distinction.

9 Customer Experience and Performance

9.1 Customer-facing decisions

The interface follows a restrained monochrome direction (white, black and neutral greys, one typeface, black primary actions, no gradients or decorative shadows), recorded as a design decision after an earlier warmer direction was rejected. Beyond appearance, the useful question was what a customer who has never seen the system will expect, misunderstand and be frustrated by. Decisions that followed:

- **Identity of what is being bought.** Variant labels, SKU and seller appear on the product page, in the cart and on the order, so the customer can see which configuration and from whom.
- **The cart survives.** The cart is stored on the server against the signed-in user or a guest cookie, so reloads and navigation do not lose it; a guest cart merges into the account on sign-in. Removing an item offers an undo. Quantity controls are clamped to available stock with an explanation, rather than failing at checkout.
- **Guest checkout is open.** Accounts add history; they do not gate purchase.
- **Coupon feedback.** Invalid, expired and under-minimum codes each get a specific message.
- **Delivery and seller visibility.** The delivery window and seller are shown at the purchase decision, in the cart and at checkout, not only after ordering.
- **Empty and failure states.** An empty cart, a search with no results (with department links), an expired checkout hold, a declined payment and a sold-out item each say what happened and what to do next. A declined payment keeps the cart and the typed address.
- **Mobile.** The product page gets a sticky purchase bar when the primary action scrolls away; filters open in a deliberate sheet; automated checks verify that key pages do not overflow horizontally on a phone viewport.
- **Accessibility.** Semantic structure, labelled form fields, announced status and error messages, visible focus and touch-sized targets were part of the design rules; a browser audit at widths from 320 to 1440 pixels was recorded in the project documents.
- **No manufactured urgency.** Stock messages reflect real stock.

Customer-perspective testing of the deployed site found three small defects, two of which were fixed before this report: a footer that still said payments were simulated while Stripe was active, and a coupon message that showed the wrong minimum spend. The third (weak results for some vague meaning queries) is described in Chapter 5. Several customer paths were *not* tested manually in this review: back and forward navigation through checkout, quantity extremes, a sell-out during checkout, and the full mobile purchase flow. Some are covered by automated tests; the rest are listed in the limitations.

9.2 Performance and scale

The architecture avoids the usual early mistakes rather than claiming scale it has not proven.

- Search and filtering run in indexed SQL; pages never load the catalogue. Result pages are bounded and paginated.
- Indexes exist for full-text and trigram search, brand, category, type, attributes (GIN), variants by product, reservations by unit, orders by owner, offers by variant, order items by order and product rating ordering.
- Facets and result pages issue a fixed number of queries independent of result size.
- Product pictures use lazy loading except for the first row, and the purchase and checkout screens are server-rendered.
- Writes that must be consistent (checkout start, payment confirmation, cancellation) are single transactions.

Measurements recorded in the repository. Early in the work the search approach was measured on PGLite before committing to PostgreSQL search. A throwaway benchmark on synthetic rows found that loading and joining the whole catalogue per query cost 63 ms at 3,000 products and 269 ms at 12,000 (and three such queries on the home page), while an indexed full-text query stayed at 1.4 ms and 3.9 ms and a trigram typo query at 2.6 ms and 8.5 ms at those sizes. After the 2,400-product build, warm requests on a production build with in-memory PGLite on a loaded laptop measured about 85–120 ms for the home page, 80–135 ms for a results page, 50–70 ms for a deep category page, about 0.2 s for a typo query and 30–50 ms for a product page. These were measured on PGLite (PostgreSQL compiled to WebAssembly), not on the deployed database, and the project documents state that they must be re-measured there. No load test was run.

Current implementation versus production scale. At Amazon's actual scale the catalogue, search, inventory, pricing and recommendation systems are separate services with their own data stores and teams. This project is one program and one primary database. It demonstrates correct behaviour at a few thousand products, not capacity at hundreds of millions; Chapter 11 describes what would change.

10 Testing, Security and Deployment

10.1 Testing and quality engineering

Table 10.1: Test layers and results at the time of writing.

Layer	Purpose	Coverage	Result
Pure unit tests	Rules with no I/O	Buy box, delivery promise, coupons, recommendation scoring, payment state function, variant resolution, concept lexicon, fusion score	Pass
Integration tests (real PostgreSQL engine)	Module behaviour through public interfaces	Reservations and last-unit race, two-phase checkout, payment flow, webhook signatures, refunds, reviews eligibility, recently viewed, sponsored placement, hybrid search, seeding	131 tests in 26 files pass
End-to-end (Playwright)	Real browser against a production build, fresh in-memory database	12 scenarios on a desktop and a phone (Pixel 7) viewport: discovery with typo recovery, purchase, variants, offers, structured search, lifecycle and cancellation, declined-card recovery, guest-to-account, overflow check, two-shopper race	Last full run: 18 run and passed, 6 skipped by design; later subset runs passed
Stripe browser test	Real Payment Element, test card and decline	Two scenarios	Skipped without keys; Not verified
Static checks	Types and boundaries	TypeScript strict; ESLint including the module-boundary rule	Clean

Tests carry stable identifiers (T1 to T132) that the decision log refers to. Representative examples: T104 starts two checkouts for the last unit concurrently and asserts that exactly one succeeds; T100 delivers a payment success after the hold expired, once when the unit is free (order placed) and once when another customer bought it (order cancelled and refunded, never double-sold); T97 and T98 show a declined payment keeping the order and hold, and a retry succeeding; T105–T109 replay, tamper with and reorder signed webhook events; T119–T121 and T132 cover coupon arithmetic, trust boundary and messages; T122 checks the buy-box tolerance and order independence; T114–T118 check that recommendations are deterministic and exclude the seed and out-of-stock items; T123 checks the verified-review rule end to end; T125 shows sponsored placement leaves organic results unchanged; T127–T131 cover the concept lexicon, fusion and fallback.

A CI workflow (lint, type check, unit and integration tests; no secrets needed) is in the repos-

itory but its first run on GitHub was not observed; the browser tests are run locally because they use the installed Chrome. One tooling lesson from development: running a development server and a production build in the same working directory corrupted an end-to-end run; it was diagnosed and avoided.

10.2 Security and trust boundaries

Only measures that exist in the code are listed. This is a demonstration project and no penetration test was performed.

- **Server-side money.** Prices, discounts, shipping, tax, totals and the payment amount are computed on the server from server-held state. The client sends a coupon *code* and quantities, nothing monetary.
- **Payment authority.** The order is placed only from a signature-verified Stripe webhook over the raw request body, or from a server-side retrieval of the `PaymentIntent`. The browser redirect and the client library's result are not trusted.
- **Secrets.** Database URL and Stripe keys exist only as environment variables; only the Stripe publishable key reaches the browser. A scan of tracked files found no keys or connection strings (only fake values in a unit test). Raw reference captures that could contain a real person's session data are excluded from the repository by ignore rules.
- **Authentication.** Passwords are hashed with `scrypt` and a per-user salt and compared in constant time; a dummy hash equalises timing for unknown emails. Session tokens are random, stored only as hashes, and sent in HTTP-only, `SameSite` cookies marked secure in production; sessions expire after 30 days.
- **Access control.** Orders and carts are scoped to their owner key; fetching another customer's order identifier returns nothing. Reviews require a delivered order.
- **Input handling.** Inputs are parsed with `Zod` at the boundary; queries use parameterised SQL through the ORM; free-text search input is normalised and bounded.
- **Idempotency and replay.** Duplicate form submissions, duplicate webhooks and repeated refunds are safe by construction (Chapter 7).

Not implemented: rate limiting, CSRF tokens beyond the framework's server-action protections, bot protection, fraud screening and audit logging.

10.3 Deployment

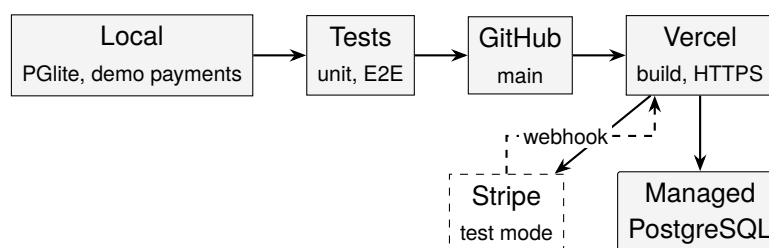


Figure 10.1: Path from development to production.

The same SQL runs everywhere: PGlite locally and in tests, a pooled PostgreSQL connection in production, selected by whether `DATABASE_URL` is set. Migrations and seeding are not run on requests; the Vercel build command applies migrations and seeds an empty database (idempotently) before building. The deployment documents choose Neon as the managed PostgreSQL host; the repository cannot confirm which host the live deployment uses. GitHub records production deployments of the final commits as successful and the site serves over HTTPS. The Stripe webhook endpoint is `/api/webhooks/stripe`; an unsigned request is rejected with HTTP 400, which was checked against the live site. Webhook delivery from Stripe to the live endpoint and the full payment were not verified. **Not verified**

11 Engineering Decisions, Limits and Future Architecture

11.1 Decisions and tradeoffs

Table 11.1: Principal decisions (numbers refer to the repository's decision log).

Decision	Alternatives considered	Why chosen	Tradeoff
PostgreSQL for search (D23)	Dedicated search engine; keeping the earlier in-memory search	Benchmarks showed in-memory scanning degrading by 3,000 products while indexed SQL stayed in milliseconds; one system to operate	Weaker relevance tuning; no learned ranking
Concept-based hybrid retrieval (D32)	Embeddings with pgvector or a hosted model	No installed model, no new paid service or secret, deterministic and testable, graceful fallback	Understands only the lexicon; some vague queries return weak results
Deterministic recommendations (D30)	Trained model	No training data in the time box; explainable and reproducible	Cannot learn from behaviour
Modular monolith (modules document)	Microservices	One deployable unit; boundaries enforced by lint; fits one developer	Boundaries are social plus tooling, not network
Provider interface for payments (D28)	Calling Stripe directly	Demo provider for tests; webhook and retrieval share one event handler	An extra abstraction to maintain
Reservation-based stock (D27)	Decrement at payment; optimistic retry	Prevents selling twice and charging for unavailable stock	Holds stock for 15 minutes from abandoned checkouts
Synthetic catalogue (D23)	Scraped or licensed product data	No data rights; deterministic and reproducible	Product data is obviously artificial
Implicit first-party offer (D26)	Every variant as explicit offers	No migration of existing cart and checkout; seller offers are additive	Two code paths for the first-party and seller cases
Derived order timeline (D24)	Stored statuses with a job runner	No scheduler needed; consistent with the clock	Time compressed for demonstration
Guest checkout open (D22)	Required accounts	Fewer abandoned purchases	Accounts add history, not security of the purchase

11.2 What was deliberately not built

- **Distributed search and learned ranking.** The catalogue is small and the ranking is rules; a dedicated engine would add infrastructure without a visible gain in the time box.
- **Machine-learned recommendation and an ad auction.** No behavioural data or traffic exists to learn from; the observable behaviour (separation, labelling, determinism) was the point.
- **Warehouse management and a real logistics network.** Inventory is a count per variant

or offer; delivery is a deterministic simulation.

- **Seller operations.** No seller portal, onboarding, payouts or seller ratings.
- **Live payments, saved cards and fraud detection.** Stripe test mode only; card data never touches the server.
- **Large-scale personalisation infrastructure.** Personalisation is a per-visitor view history.
- **Reviews beyond the basics.** No helpful votes, images or moderation.
- **Email, returns, internationalisation, tax by jurisdiction.** A single country and provisional tax and shipping rules.

Each was deferred because it either needed infrastructure or data that the time box could not supply or would not change what a reviewer could verify about the system's correctness.

11.3 Limitations

Product. The catalogue, sellers, reviews and campaigns are synthetic and the pictures are generated illustrations. Some vague meaning queries return weak results. The expired-coupon message has no date. Taxes and shipping are demo rules.

Infrastructure. One application and one primary database; no cache layer, queue, search service, observability stack or rate limiting. Reservation expiry is lazy, which is correct for availability but means abandoned holds are not actively cleaned.

Scale. Measured behaviour is for a few thousand products on PGlite and a managed database of modest size; no load test was run and the recorded timings were not repeated on the deployed database.

Validation. A completed live Stripe test payment and webhook delivery were not confirmed from the repository; the Stripe browser tests were skipped for lack of keys; the first CI run on GitHub was not observed; several customer paths (back-navigation through checkout, mid-checkout sell-out, full mobile purchase) were not manually verified in a final pass.

11.4 Future architecture

Label

This section describes possible evolution. None of it exists in the repository.

If the prototype became a product, the natural seams are already the module boundaries:

- **Catalogue and search services.** Move catalogue ingestion and indexing behind a service with a search engine for relevance and facets; keep PostgreSQL as the source of truth. A learned retriever (embeddings) could replace the concept lexicon, with the lexical path retained as the fallback exactly as today.
- **Inventory service.** Extract reservations into a service with per-location stock and expiry workers, preserving the held, committed, released lifecycle.
- **Payments and order management.** Keep the provider interface; add live-mode controls, asynchronous reconciliation against the provider and a returns and refund workflow.

- **Fulfilment and delivery.** Replace the deterministic promise with carrier and warehouse data, keeping the same function signature as the contract.
- **Recommendation and ads platforms.** Train ranking from behavioural events, run experiments, and add an auction with budgets and reporting, retaining the rule that sponsored content is separate and labelled.
- **Seller platform, analytics and experimentation.** Seller onboarding and ratings; event pipelines and A/B testing.

12 Final Engineering Takeaways

- **The frontend is the visible surface.** The pages that look like a shop are the output of a catalogue model, offers, reservations, a payment lifecycle and ranking rules. Most of the effort, and nearly all of the risk, sat below the interface.
- **Domain modelling pays for itself.** Separating product, variant and offer made variants, sellers, the buy box, the cart and order snapshots fall out of one idea rather than becoming special cases.
- **State transitions need an owner.** Treating inventory, payment and order as three machines, and letting only verified events move the payment machine, removed a class of bugs (double sale, order without payment, refund claimed but not made) by construction.
- **Customer experience depends on backend correctness.** Clear messages for an expired hold, an under-minimum coupon or a declined card are only possible because the backend returns distinct, typed outcomes.
- **Deterministic systems are often the right choice under a deadline.** Rules for ranking, recommendations, delivery and concept search are explainable, reproducible and testable; their limits are known and were written down instead of hidden.
- **Scope management is an engineering skill.** What was left out (Chapter 11) was chosen deliberately, and the order of work protected the purchase path first.
- **Reliability beats feature count.** A smaller set of features, each with tests and an honest statement of what is and is not verified, is more useful than a longer list that cannot be trusted. The unverified items in this report, chiefly a completed live Stripe payment, are the most important follow-up work.